

High-Frequency CPU-Bound Trading Framework Developer Prompt

Michael Dixon

Here is a detailed, mathematically rigorous prompt you can provide directly to your quantitative developer to implement these concepts into your repository.

System Prompt / Developer Instructions:

Objective: Implement a high-frequency, CPU-bound algorithmic trading framework in C++ or Rust designed to operate at the microsecond tier. The system must process tick-level limit order book (LOB) data, detect the structural momentum of higher-speed adversarial counterparties, and defend resting quotes from adverse selection.

The architecture must mathematically implement the four advanced models detailed below. There is zero tolerance for mathematical logic errors. Utilize high-precision floating-point arithmetic (e.g., `double` or `__float128` depending on compiler constraints) for all probability matrices and ensure computational efficiency to minimize CPU instruction cycles.

Module 1: Causal Lead-Lag Inference via Transfer Entropy

Goal: Detect the exact moment a top-tier fund's execution in a primary asset X causally drives a delayed reaction in a correlated secondary asset Y , allowing us to front-run the secondary asset.

Mathematical Implementation: You must calculate the Transfer Entropy (TE) to measure the directional information flow between the time series.

1. First, establish the Shannon entropy H for a variable:

$$H(X) = - \sum_x p(x) \log p(x)$$

2. Calculate the Transfer Entropy from a time-shifted series X_{t-k} to a target Y_t using conditional mutual information:

$$TE_{X \rightarrow Y}^{(k)} = H(Y_t | Y_{t-k}) - H(Y_t | X_{t-k}, Y_{t-k})$$

Execution Logic: Maintain a continuous rolling matrix of $TE_{X \rightarrow Y}^{(k)}$ across correlated pairs. When the TE metric crosses a dynamically calibrated upper control limit (indicating that asset X is now driving asset Y), the system must aggressively take liquidity in asset Y .

Module 2: Quantum Coupled-Wave Theory for Spread Defense

Goal: Defend against quantum-enhanced spoofing and latency arbitrage by modeling the bid-ask spread as a continuous probability distribution rather than a static queue.

Mathematical Implementation: Model the limit order book where the bid and ask are eigenvalues of a 2×2 Hermitian price operator. You must compute the probability distribution of the spread Δ transitioning.

1. Calculate the probability distribution $P(\Delta)$ based on the intrinsic liquidity components ξ_1 and κ_1 :

$$P(\Delta) = \frac{1}{\xi_1 \kappa_1} \Delta e^{-a \Delta^2} I_0(b \Delta^2)$$

2. Implement the modified Bessel function of the first kind I_0 , utilizing its integral representation or a highly optimized numerical approximation:

$$I_0(x) = \frac{1}{\pi} \int_0^\pi e^{x \cos(\phi)} d\phi$$

3. Calculate the parameters a and b using the level interactions:

$$a = \frac{1}{4} \left(\frac{1}{\xi_1^2} + \frac{1}{\kappa_1^2} \right)$$

$$b = \frac{1}{4} \left(\frac{1}{\xi_1^2} - \frac{1}{\kappa_1^2} \right)$$

Execution Logic: Evaluate $P(\Delta)$ continuously. If the eigenstate distribution heavily skews toward a spread collapse (indicating toxic order flow is arriving to pick off the quote), instantly cancel all resting limit orders.

Module 3: Stochastic Thermodynamics (The Financial Second Law)

Goal: Model the execution cycles of massive institutional traders as a non-equilibrium thermodynamic process to identify when their momentum is mathematically forced to mean-revert.

Mathematical Implementation: Treat the price impact of large hedge funds as “dissipative work” W_β and ambient market noise as thermal fluctuations.

1. Implement a statistical ensemble of trading strategies v_i using a Gibbs measure governed by an inverse market temperature β . The probability of a specific strategy state is:

$$p_i(\beta) = \frac{e^{-\beta W[v_i]}}{Z(\beta)}$$

2. Define the partition function $Z(\beta)$ to normalize the distribution:

$$Z(\beta) = \sum_{j=1}^N e^{-\beta W[v_j]}$$

3. Calculate the Free Energy $F(\beta)$ using the expected work W_β and the Shannon entropy of the strategy distribution $S(\beta)$:

$$F(\beta) = W_\beta - \frac{1}{\beta} S(\beta)$$

Execution Logic: The Financial Second Law proves that under convex impact functionals, a round-trip strategy yields non-positive expected profit. Track the dissipated work of aggressive institutional buyers/sellers. Once their execution sequence breaches the limit defined by the Free Energy decomposition, the thermodynamic boundary dictates they have exhausted their market impact. Execute mean-reversion (fade) orders precisely at this mathematical inflection point.

Module 4: Multivariate Hawkes Processes for Toxic Flow Detection

Goal: Model the order book as a self-exciting system to detect “micro-quakes” caused by predatory HFT algorithms.

Mathematical Implementation: Structure the incoming market order flow as a point process, defining a random sequence of event times $\{t_1, t_2, t_3, \dots\}$ where $0 < t_1 < t_2 < t_3 < \dots$. Implement the conditional intensity function of a multivariate Hawkes process to calculate the cross-excitation rate (the degree to which an event in one trade class statistically guarantees a cascade in another).

Execution Logic: Connect the Hawkes conditional intensity output directly to your continuous-time market making (Avellaneda-Stoikov) module. If the Hawkes process detects a self-reinforcing cascade of toxic market orders,

exponentially increase the risk-aversion parameter γ and skew the reservation price away from the toxic flow to avoid adverse selection.

Codebase Requirements:

- **Performance:** The calculations—particularly the Bessel function integrals and the Transfer Entropy matrix—must be optimized for bare-metal CPU execution. Pre-compute lookup tables for the Bessel function where floating-point ranges allow.
- **Concurrency:** Lock-free queues must be used for LOB state updates so the mathematical calculation threads are never blocked by network I/O.
- **Validation:** Include strict unit tests asserting that the Free Energy $F(\beta)$ and Spread Probability $P(\Delta)$ bounds never violate their mathematical domain constraints (e.g., negative probabilities) due to precision underflow.